

Esta página ha sido traducida del inglés por la comunidad. Aprende más y únete a la comunidad de MDN Web Docs.

## if...else

**Baseline** Widely available

La sentencia `if...else` ejecuta una sentencia, si una condición especificada es evaluada como [verdadera](#). Si la condición es evaluada como [falsa](#) <sup>(inglés)</sup>, otra sentencia en la cláusula opcional `else` será ejecutada.

## Pruébalo

JavaScript Demo: Statement - If...Else

```
1 function testNum(a) {  
2   let result;  
3   if (a > 0) {  
4     result = 'positive';  
5   } else {  
6     result = 'NOT positive';  
7   }  
8   return result;  
9 }  
10  
11 console.log(testNum(-5));  
12 // Expected output: "NOT positive"  
13
```



## Sintaxis

JS

```
if (condición)  
  sentencia1
```

// Con una cláusula else

```

if (condición)
  sentencia1
else
  sentencia2

```

condición

Una expresión que puede ser evaluada como [verdadera](#) o [falsa](#) <sup>(inglés)</sup>.

sentencia1

Sentencia que se ejecutará si `condición` es evaluada como [verdadera](#). Puede ser cualquier sentencia, incluyendo otras sentencias `if` anidadas. Para ejecutar múltiples sentencias, use una sentencia [block](#) (`{ ... }`) para agruparlas. Para no ejecutar ninguna sentencia, usa una sentencia [vacía](#).

sentencia2

Sentencia que se ejecutará si `condición` se evalúa como [falsa](#) <sup>(inglés)</sup>, y existe una cláusula `else`. Puede ser cualquier sentencia, incluyendo sentencias *block* y otras sentencias `if` anidadas.

## Descripción

Múltiples sentencias `if...else` pueden ser anidadas para crear una cláusula `else if`. Note que no hay una palabra clave `elseif` (en una sola palabra) en JavaScript.

```

if (condición1)
  sentencia1
else if (condición2)
  sentencia2
else if (condición3)
  sentencia3
//...
else
  sentenciaN

```

Para entender como esto funciona, así es como se vería si el anidamiento hubiera sido indentado correctamente:

```

if (condición1)
  sentencia1
else
  if (condición2)
    sentencia2
  else
    if (condición3)
      ...

```

Para ejecutar varias sentencias en una cláusula, use una sentencia *block* (`{ /* ... */ }`) para agruparlas.

JS

```
if (condición) {  
  sentencia1;  
} else {  
  sentencia2;  
}
```

No usar *blocks* puede ocasionar un comportamiento inesperado, especialmente si el código es estructurado manualmente. Por ejemplo:

JS

```
function checkValue(a, b) {  
  if (a === 1)  
    if (b === 2)  
      console.log("a is 1 and b is 2");  
  else  
    console.log("a is not 1");  
}
```

Este código puede parecer inocente — sin embargo, si ejecutamos `checkValue(1, 3)` registrará el mensaje "a is not 1". Esto debido a que en el caso de [dangling else](#), la cláusula `else` se conectará a la cláusula `if` más cercana. Por lo tanto, el código anterior, indentado apropiadamente, se vería así:

JS

```
function checkValue(a, b) {  
  if (a === 1)  
    if (b === 2)  
      console.log("a is 1 and b is 2");  
  else  
    console.log("a is not 1");  
}
```

Generalmente, es una buena práctica usar siempre sentencias block, especialmente en código que incluya sentencias if anidadas.

JS

```
function checkValue(a, b) {  
  if (a === 1) {  
    if (b === 2) {  
      console.log("a is 1 and b is 2");  
    }  
  } else {  
    console.log("a is not 1");  
  }  
}
```

No confundir los valores booleanos primitivos `true` y `false` con los valores verdadero y falso del objeto [Boolean](#). Cualquier valor diferente de `undefined`, `null`, `0`, `-0`, `NaN`, o la cadena vacía (`""`), y cualquier objeto, incluso un objeto Boolean cuyo valor es `false`, se evalúa como [verdadero](#) en una sentencia condicional. Por ejemplo:

JS

```
const b = new Boolean(false);
// Esta condición se evalúa como verdadera
if (b) {
  console.log("b is truthy"); // "b is truthy"
}
```

## Ejemplos

### Uso de `if...else`

Note que no hay sintaxis `elseif` en JavaScript. Sin embargo, puede escribirse con un espacio entre `else` y `if`:

JS

```
if (cipherChar === fromChar) {
  result += toChar;
  x++;
} else {
  result += clearChar;
}
```

### Using `else if`

Note que no hay sintaxis `elseif` en JavaScript. Sin embargo, puede escribirse con un espacio entre `else` y `if`:

JS

```
if (x > 50) {
  /* hace algo */
} else if (x > 5) {
  /* hace algo */
} else {
  /* hace algo */
}
```

### Asignación en una expresión condicional

Casi nunca deberías tener un `if...else` con una asignación `x = y` como condición:

JS

```
if ((x = y)) {  
  // ...  
}
```

Porque a diferencia de los bucles [while](#), la condición es evaluada sólo una vez, así que la asignación es ejecutada una vez. El código anterior es equivalente a:

JS

```
x = y;  
if (x) {  
  // ...  
}
```

El cual es mucho más claro. Sin embargo, en el raro caso que te encuentres en la situación de hacer algo como eso, la documentación del bucle [while](#) tiene una sección llamada [Usando una asignación como una condición](#) con nuestras recomendaciones.

## Especificaciones

### Specification

[ECMAScript Language Specification](#)

[# sec-if-statement](#)

## Compatibilidad con navegadores

[Report problems with this compatibility data on GitHub](#)

|           | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Internet | WebView Android | WebView on iOS | Deno | Node.js |
|-----------|--------|------|---------|-------|--------|----------------|---------------------|---------------|---------------|------------------|-----------------|----------------|------|---------|
| if...else | 1      | 12   | 1       | 3     | 1      | 18             | 4                   | 10.1          | 1             | 1.0              | 4.4             | 1              | 1.0  | 0.10.0  |

Tip: you can click/tap on a cell for more information.

Full support

## Véase También

- [block](#)
- [switch](#)

### Help improve MDN

Was this page helpful to you?

[Learn how to contribute.](#)

This page was last modified on 28 ago 2024 by [MDN contributors](#).

